ICON

Industrial Diagnosis

Sistema intelligente per la diagnosi simbolica e la valutazione ML dei guasti industriali

Autore: Antonio De Rosa [MAT. 762940]

Corso: Ingegneria della Conoscenza

Link GitHub: ICON-25-26

Anno accademico: 2025-2026

Relazione tecnica del progetto

Sommario

Capitolo 1 – Introduzione

1.1 Dominio di interesse

1.2 Sommario del KBS

1.3 Requisiti funzionali e strumenti

1.4 Argomenti di interesse

Capitolo 2 – Rappresentazione della Conoscenza (Ontologie)

2.1 Ingegnerizzazione dell'Ontologia e Scelte Implementative

2.2 Definizione della TBox: Tassonomia delle Classi e Gerarchie

2.3 Object Properties: Relazioni, Domini e Codomini

2.4 Restrizioni Logiche ed Assiomi (Description Logic)

2.5 Definizione della ABox: Istanze, Topologia e Sintomi Grezzi

2.6 Dimensioni e complessità della Knowledge Base

Capitolo 3 – Ragionamento Automatico

3.1 Il Paradigma del Ragionamento Automatico nel KBS

3.2 Il Motore di Inferenza HermiT e le Logiche Descrittive

3.3 Implementazione del Modulo semantic_reasoner.py

3.4 Meccanismo di Materializzazione dei Sintomi ed Estrazione Classi Indirette

3.5 Ottimizzazione delle Prestazioni: Il Sistema di Caching Semantico

Capitolo 4 – Apprendimento e Incertezza (OntoBK)

4.1 Incertezza nel Dominio e Modellazione Multimodale dei Dati

4.2 Il Paradigma OntoBK e il Semantic Lifting

4.3 Pre-processing dello Spazio delle Feature: Standardizzazione e Codifica Nominale

4.4 Il Modello Predittivo Ensemble: Random Forest

4.5 Classificazione a Massimizzazione del Margine: Support Vector Machines (SVM)

4.6 Il Paradigma Connessionista: Reti Neurali Artificiali (Multi-Layer Perceptron)

4.7 Valutazione Statistica Rigorosa e Analisi Comparativa delle Prestazioni

4.8 Diagnosi simbolica, interpretabilità e gestione delle evidenze

Capitolo 5 – Conclusioni e Sviluppi Futuri

5.1 Sintesi dei Risultati Raggiunti

5.2 Valutazione Critica dell'Architettura

5.3 Sviluppi Futuri

Capitolo 1 – Introduzione

1.1 Dominio di interesse

Il progetto ICON — Industrial Diagnosis affronta il problema della diagnosi preliminare dei guasti in sistemi industriali. In un impianto reale, un'anomalia può manifestarsi attraverso misure numeriche (ad esempio temperatura, corrente, vibrazioni, pressione e portata) e attraverso sintomi descritti in forma categoriale. La stessa evidenza può essere compatibile con più guasti; inoltre, non tutti i sensori sono disponibili su ogni macchina e alcune osservazioni possono essere sconosciute o non applicabili.

Il sistema considera quattro tipologie di macchina: motore elettrico, pompa industriale, compressore e nastro trasportatore. Le ipotesi diagnostiche sono raggruppate in quattro famiglie: guasto termico, meccanico, elettrico e idraulico. La famiglia idraulica è modellata come compatibile soltanto con pompa industriale e compressore, coerentemente con la struttura semplificata scelta per il dominio.

Il progetto non si propone come strumento certificato per la manutenzione di impianti reali. Il dataset utilizzato è sintetico e le probabilità che guidano la generazione dei casi sono illustrative: consentono di sviluppare e confrontare metodi in modo riproducibile, ma non sostituiscono misure raccolte sul campo né una validazione industriale.

1.2 Sommario del KBS

Il Knowledge-Based System (KBS) integra una base di conoscenza OWL, un motore di inferenza e una componente di apprendimento automatico. Le due modalità sono complementari, ma non vanno confuse: la diagnosi simbolica produce un ordinamento di ipotesi in base al supporto delle evidenze e ai vincoli di compatibilità; la valutazione ML confronta classificatori addestrati su feature grezze e su una rappresentazione arricchita con informazioni semantiche.

1. Costruzione dell'ontologia che descrive macchine, componenti, sensori, sintomi, osservazioni e famiglie di guasto.
2. Esecuzione del reasoner HermiT per classificare semanticamente gli individui in base agli assiomi OWL ed esportazione dell'ontologia, con le classificazioni inferite, in results/industrial_diagnosis.owl.
3. Acquisizione di una macchina e dello stato dei sintomi nella diagnosi simbolica.
4. Calcolo del supporto simbolico per i guasti compatibili, distinguendo evidenze favorevoli, sintomi associati ma dichiarati assenti e stati ignorati.
5. Confronto sperimentale di Random Forest, SVM RBF e MLP su rappresentazione Base e rappresentazione OntoBK.

1.3 Requisiti funzionali e strumenti

Requisito	Realizzazione
R1 — Modellazione della conoscenza	Ontologia OWL costruita in Python tramite Owlready2.
R2 — Inferenza automatica	Reasoner HermiT invocato tramite sync_reasoner.
R3 — Gestione delle evidenze	Stati presente, assente, sconosciuto e non applicabile.
R4 — Diagnosi simbolica	Filtraggio per compatibilità macchina-guasto e conteggio di sintomi di supporto.
R5 — Apprendimento supervisionato	Pipeline scikit-learn con Random Forest, SVM RBF e MLP.
R6 — Confronto	Stessa procedura di validazione per le due rappresentazioni.

Base/OntoBK	
R7 — Riproducibilità	Dataset sintetico generabile con seed, risultati CSV e test automatici.

Gli strumenti principali sono Python, Owlready2, un'installazione Java compatibile con HermiT, pandas, NumPy e scikit-learn. I test sono realizzati con pytest. I file di output comprendono l'ontologia esportata, i risultati comparativi e le matrici di confusione. Il punto di ingresso è il modulo src/main.py, avviabile dalla directory principale con il comando seguente:

```
python -m src.main
```

1.4 Argomenti di interesse

Argomenti del corso su cui si basa il progetto e modo in cui compaiono nell'implementazione.

- **Sistemi basati su conoscenza.** Il sistema è un piccolo Knowledge-Based System: una base di conoscenza esplicita (ontologia OWL), un meccanismo di ragionamento (HermiT e diagnosi simbolica) e una componente di apprendimento dai dati. Nello spazio di progettazione è un sistema a orizzonte statico (una diagnosi per caso), modulare (livello simbolico e statistico distinti), con rappresentazione per individui e relazioni e apprendimento supervisionato.
- **Ontologie e rappresentazione relazionale della conoscenza.** La conoscenza è modellata come ontologia OWL, con classi gerarchiche, individui collegati da relazioni (has_component, indicates_fault, compatible_with_machine) e proprietà dei dati. L'ontologia è serializzata in RDF/XML, quindi come insieme di triple (soggetto, predicato, oggetto) che descrivono individui e relazioni: la stessa struttura su cui si basano i knowledge graph. Gli assiomi di equivalenza, espressi in logica descrittiva (restrizione esistenziale), permettono a HermiT di classificare automaticamente i sintomi. La diagnosi simbolica risale dai sintomi osservati alle famiglie di guasto che possono spiegarli, ordinando le ipotesi in base alle evidenze.
- **Gestione dell'informazione incompleta.** Gli stati PRESENT, ABSENT, UNKNOWN e NON_APPLICABLE distinguono un'evidenza contraria dalla semplice mancanza di informazione, in linea con la semantica di OWL: uno stato sconosciuto non viene mai trattato come assenza.
- **Apprendimento supervisionato e reti neurali.** Classificazione a quattro classi con Random Forest (bagging di alberi decisionali), SVM con kernel RBF e rete feed-forward MLP (strati nascosti da 50 e 25 unità, early stopping). La valutazione usa cross-validation stratificata ripetuta (5 fold × 3 ripetizioni), metriche macro e matrici di confusione.
- **Integrazione fra conoscenza e apprendimento (OntoBK).** Il confronto fra rappresentazione Base e rappresentazione OntoBK misura se feature derivate dalla base di conoscenza aiutino i classificatori. L'esito è valutato empiricamente, anche quando non favorevole, e discusso criticamente.

Capitolo 2 – Rappresentazione della Conoscenza (Ontologie)

2.1 Ingegnerizzazione dell'Ontologia e Scelte Implementative

L'ontologia viene costruita programmaticamente in src/ontology.py attraverso Owlready2. La scelta permette di mantenere gli assiomi vicino al codice, ricostruire la base di conoscenza in modo riproducibile ed esportarla in formato RDF/XML. L'IRI configurato nel prototipo è https://example.org/icon/industrial-diagnosis.owl: è un identificatore dimostrativo, da sostituire con un IRI stabile se l'ontologia verrà pubblicata come risorsa web.

Il modello separa concetti di dominio (classi), relazioni fra individui (object properties) e attributi letterali delle osservazioni (data properties). La separazione rende possibile esprimere vincoli semantici indipendenti dalla logica procedurale del programma Python.

Elemento	Scopo
Machine / MachineType	Rappresenta una macchina e i suoi tipi.
Component	Rappresenta componenti fisici rilevanti per la diagnosi.
Sensor	Rappresenta sensori associati alle macchine e ai componenti monitorati.
Observation	Classe predisposta per rappresentare una rilevazione; nella versione corrente non esistono individui Observation e la classe non è usata da diagnosi né ML.
Symptom	Rappresenta un sintomo diagnostico osservabile.
Fault / FaultFamily	Organizza le ipotesi diagnostiche per famiglia.

2.2 Definizione della TBox: Tassonomia delle Classi e Gerarchie

La TBox contiene le classi e gli assiomi che definiscono il vocabolario del dominio. La gerarchia principale distingue Machine, Component, Sensor, Observation, Symptom e Fault. FaultFamily è una sottoclasse di Fault e si specializza in ThermalFault, MechanicalFault, ElectricalFault e HydraulicFault. MachineType specializza Machine e comprende ElectricMotor, IndustrialPump, Compressor e ConveyorBelt.

Sono inoltre definite sottoclassi per categorie di sensori (TemperatureSensor, CurrentSensor, VibrationSensor, PressureSensor e FlowSensor) e per componenti (MotorComponent, Bearing, CoolingSystem, ElectricalSupply, HydraulicCircuit e BeltDrive). Le classi sintomatiche ThermalSymptom, MechanicalSymptom, ElectricalSymptom e HydraulicSymptom vengono definite attraverso assiomi di equivalenza, in modo da poter essere riconosciute dal reasoner sulla base delle relazioni esplicite.

La gerarchia è volutamente compatta: il progetto dimostra come usare tassonomia e assiomi per derivare classificazioni, senza pretendere di rappresentare tutte le componenti e modalità di guasto presenti in un impianto industriale complesso.

2.3 Object Properties: Relazioni, Domini e Codomini

Le object properties collegano gli individui del dominio. Le relazioni implementate sono riportate nella tabella. Nel codice non sono dichiarate proprietà transitive o inverse: pertanto la relazione non va descritta come transitiva né si deve presumere che il reasoner generi automaticamente relazioni inverse non assiomatizzate. Il ragionamento deriva invece dalle gerarchie e dagli assiomi effettivamente dichiarati.

Proprietà	Dominio → Codominio	Significato
has_component	Machine → Component	La macchina include un componente.
has_sensor	Machine → Sensor	Il sensore è associato alla macchina.
monitors_component	Sensor → Component	Il sensore monitora un componente.
observes_symptom	Observation → Symptom	L'osservazione rileva un sintomo.

concerns_machine	Observation → Machine	L'osservazione riguarda una macchina.
indicates_fault	Symptom → FaultFamily	Il sintomo è associato a una famiglia di guasto.
affects_component	FaultFamily → Component	La famiglia di guasto interessa un componente.
has_observation	Machine → Observation	La macchina ha un'osservazione.
compatible_with_machine	FaultFamily → MachineType	La famiglia è compatibile con un tipo di macchina.

Le data properties observation_state, measurement_value e measurement_unit descrivono rispettivamente lo stato dell'osservazione, il valore numerico e l'unità di misura. Sono dichiarate funzionali, cioè ciascuna osservazione ha al più un valore per la specifica proprietà funzionale.

Le proprietà has_observation, observes_symptom e concerns_machine e le tre data properties sono dichiarate nello schema ma non sono popolate né usate dalla diagnosi o dall'estrazione delle feature: sono una struttura predisposta per estensioni future. Le relazioni effettivamente operative sono indicates_fault (Symptom → FaultFamily), che fonda la classificazione semantica dei sintomi e il supporto diagnostico, e compatible_with_machine (FaultFamily → MachineType), che fonda il filtro di compatibilità.

2.4 Restrizioni Logiche ed Assiomi (Description Logic)

Le definizioni equivalenti dei sintomi usano una restrizione esistenziale. Per esempio, ThermalSymptom è equivalente all'intersezione fra Symptom e l'insieme degli individui che sono collegati tramite indicates_fault ad almeno un individuo di ThermalFault. In notazione descrittiva:

```
ThermalSymptom ≡ Symptom ⊓ ∃ indicates_fault.ThermalFault
```

Lo stesso schema viene applicato alle famiglie meccanica, elettrica e idraulica. In presenza degli assiomi e delle relazioni necessarie, HermiT può inferire che un individuo sintomo appartenga alla corrispondente classe specializzata anche se tale appartenenza non è stata asserita direttamente.

Le proprietà hanno domain e range espliciti. In OWL, domain e range agiscono come assiomi inferenziali: un individuo collegato tramite una proprietà può essere classificato come appartenente al dominio o al codominio, non sono semplicemente controlli di validazione. La compatibilità fra famiglie di guasto e tipi di macchina viene inoltre usata dalla procedura diagnostica per escludere ipotesi non applicabili alla macchina selezionata.

2.5 Definizione della ABox: Istanze, Topologia e Sintomi Grezzi

La ABox comprende individui concreti che popolano le classi. Nel modello sono presenti individui rappresentativi delle quattro famiglie di guasto, dei quattro tipi di macchina, dei componenti, dei sensori e dei sintomi. Fra i sintomi figurano overheating, abnormal_vibration, current_anomaly, pressure_anomaly, flow_anomaly, unusual_noise e reduced_performance.

Le relazioni asserite collegano macchine e componenti, sensori e componenti monitorati, sintomi e famiglie di guasto, oltre alla compatibilità fra guasti e macchine. Queste asserzioni formano la topologia della base di conoscenza su cui lavora il reasoner. Il dataset tabellare contiene invece i casi sintetici e gli stati delle evidenze: non ogni riga del CSV viene convertita automaticamente in un individuo OWL distinto durante la diagnosi simbolica.

Questa distinzione è importante: l'ontologia codifica la conoscenza generale del dominio, mentre il dataset serve principalmente alla valutazione supervisionata. Il collegamento fra i due livelli avviene attraverso i nomi mappati in SemanticFeatureExtractor e attraverso le relazioni simboliche lette dalla base OWL.

2.6 Dimensioni e complessità della Knowledge Base

La complessità della KB è valutata in termini strutturali e non come complessità computazionale formale del ragionamento. Lo schema contiene 31 classi, 9 object properties e 3 data properties funzionali, con 4 assiomi di equivalenza per la classificazione dei sintomi. La ABox contiene 26 individui: 4 famiglie di guasto, 4 tipi di macchina, 6 componenti, 5 sensori e 7 sintomi. I valori sono verificabili in src/ontology.py e tramite il riepilogo disponibile nel menu del programma.

Per un singolo caso il ranking diagnostico esamina un numero finito di sintomi (7) e di famiglie di guasto (4): il suo costo cresce linearmente con entrambi. Questa osservazione riguarda la procedura di ranking e non sostituisce un'analisi della complessità del reasoner OWL. La KB è un modello didattico circoscritto, non una tassonomia esaustiva dei guasti industriali.

Capitolo 3 – Ragionamento Automatico

3.1 Il Paradigma del Ragionamento Automatico nel KBS

Il ragionamento automatico consente di ricavare conoscenza implicita dagli assiomi della base. Nel progetto il reasoner OWL classifica gli individui sintomo e rende disponibili le classi inferite. A valle di tale inferenza, la diagnosi simbolica applica una procedura esplicita di confronto delle evidenze: non è un calcolo probabilistico e non restituisce una probabilità calibrata del guasto.

Il flusso operativo è: costruzione dell'ontologia, esecuzione di HermiT, esportazione dell'ontologia, acquisizione della macchina e degli stati dei sintomi, applicazione dei vincoli di compatibilità, conteggio delle evidenze e restituzione delle ipotesi ordinate. I due livelli — inferenza OWL e ranking diagnostico procedurale — sono collegati ma distinti.

3.2 Il Motore di Inferenza HermiT e le Logiche Descrittive

HermiT è un reasoner OWL 2 basato su logiche descrittive. Nel progetto viene invocato tramite owlready2.sync_reasoner sul mondo OWL creato da build_ontology(). L'opzione infer_property_values=True richiede anche l'inferenza di valori di proprietà supportati dal reasoner. Le classificazioni risultanti sono poi interrogate attraverso INDIRECT_is_a, che permette di accedere alle appartenenze dirette e indirette dell'individuo.

L'esecuzione del reasoner è necessaria prima di interrogare le classi inferite. Il modulo controlla tale prerequisito tramite reasoner_run e solleva un errore se si tenta di accedere alle classi senza aver eseguito l'inferenza. Questo rende esplicito il ciclo di vita del componente e riduce il rischio di utilizzare una cache non inizializzata.

3.3 Implementazione del Modulo semantic_reasoner.py

DiagnosticReasoner incapsula la gestione del reasoner e la procedura di diagnosi simbolica. Le sue responsabilità principali sono l'esecuzione dell'inferenza, il recupero degli individui OWL, l'estrazione delle classi inferite e l'elaborazione degli stati dei sintomi.

- FAULTS mappa le etichette del dataset ai nomi degli individui OWL delle famiglie di guasto.
- SYMPTOMS definisce il vocabolario dei sintomi riconosciuti dal sistema.
- VALID_STATES documenta gli stati ammessi dalla diagnosi simbolica: PRESENT, ABSENT, UNKNOWN e NON_APPLICABLE.
- get_inferred_classes_for_symptom controlla l'avvenuta inferenza, verifica che l'individuo sia un sintomo ed estrae le classi indirette pertinenti.
- diagnose applica la compatibilità macchina-guasto, raccoglie le evidenze favorevoli e segnala i sintomi associati dichiarati assenti.

Il supporto di una diagnosi è un conteggio simbolico. Non deve essere interpretato come confidenza statistica: due guasti con supporto 3 non hanno necessariamente probabilità del 50%, e il conteggio non considera esplicitamente la frequenza empirica dei guasti né la dipendenza fra sintomi.

3.4 Meccanismo di Materializzazione dei Sintomi ed Estrazione Classi Indirette

Gli assiomi di equivalenza permettono di classificare indirettamente gli individui sintomo. Per esempio, se un sintomo è collegato tramite indicates_fault a un individuo di ThermalFault, la definizione di ThermalSymptom

rende possibile inferire l'appartenenza alla classe. L'estrazione avviene interrogando le classi indirette dell'individuo e rimuovendo le classi generiche escluse (Thing, Symptom, Fault e FaultFamily).

La materializzazione è qui intesa come disponibilità delle classificazioni inferite nell'ambiente OWL dopo l'esecuzione del reasoner. Non si tratta di un processo di apprendimento dei nuovi assiomi né di aggiornamento automatico della conoscenza a partire dai risultati ML.

Nel file results/industrial_diagnosis.owl, esportato dopo l'esecuzione di HermiT, i sintomi risultano con le classi inferite: per esempio overheating è classificato come ThermalSymptom, MechanicalSymptom ed ElectricalSymptom, perché indicates_fault lo collega ai tre guasti corrispondenti.

3.5 Ottimizzazione delle Prestazioni: Il Sistema di Caching Semantico

DiagnosticReasoner mantiene cache_inferenze, un dizionario che associa il nome di ciascun sintomo alle classi inferite già estratte. Dopo una nuova esecuzione di run_inference(), la cache viene svuotata per evitare di riutilizzare risultati potenzialmente non più validi. Se la richiesta riguarda un sintomo già presente nella cache, il modulo restituisce una copia della lista memorizzata invece di ripetere la ricerca e l'estrazione.

La cache riduce il lavoro ripetuto nelle chiamate successive, soprattutto quando la stessa ontologia viene utilizzata per più casi o durante l'estrazione delle feature semantiche. Non sostituisce HermiT e non evita il costo della prima inferenza: ottimizza le interrogazioni ripetute successive.

Capitolo 4 – Apprendimento e Incertezza (OntoBK)

4.1 Incertezza nel Dominio e Modellazione Multimodale dei Dati

La diagnosi industriale è caratterizzata da misure numeriche variabili, sensori mancanti, osservazioni rumorose e sintomi non sempre disponibili. Il dataset industrial_faults.csv contiene 3.000 casi sintetici e 22 colonne, compresa la variabile target fault. Le variabili descrittive comprendono tipo di macchina, frazione di carico, ore operative, misure di temperatura/corrente/vibrazione/pressione/portata, stati dei sensori e stati dei sintomi.

La generazione usa valori di riferimento dipendenti dal tipo di macchina, effetti illustrativi per ciascuna famiglia di guasto, rumore casuale e probabilità configurate per i sintomi. Gli stati sconosciuti e le misure mancanti sono introdotti con probabilità impostabili. La generazione con random_state fisso rende ripetibile la costruzione del dataset.

Variabile / gruppo	Descrizione
machine_type	Motore elettrico, pompa industriale, compressore, nastro trasportatore.
load_fraction, operating_hours	Carico relativo e ore operative.
temperature_c, current_a, vibration_mm_s	Misure numeriche di temperatura, corrente e vibrazione.
pressure_bar, flow_rate_l_min	Pressione e portata; non applicabili a tutte le macchine.
*_status	Stato di disponibilità o mancanza del sensore.
symptom_*	Stato dei sette sintomi: presente, assente, sconosciuto o non applicabile.
fault	Classe target a quattro valori.

Il dataset è costruito per rendere possibile il confronto metodologico. Poiché la generazione usa effetti e probabilità specificati a priori, i risultati misurano quanto i modelli recuperino i pattern simulati e non dimostrano direttamente prestazioni su impianti reali.

4.2 Il Paradigma OntoBK e il Semantic Lifting

L'approccio OntoBK arricchisce la rappresentazione tabellare con feature derivate dalla base di conoscenza. La rappresentazione Base include feature numeriche, codifica del tipo di macchina e codifica degli stati dei sensori e dei sintomi. La rappresentazione OntoBK conserva le feature Base e aggiunge informazioni semantiche.

SemanticFeatureExtractor recupera le classi inferite per i sintomi e legge dall'ontologia le relazioni fra sintomi e famiglie di guasto e la compatibilità fra guasti e macchine. Le 20 feature aggiuntive rispetto alla rappresentazione Base (59 colonne contro 79) sono: quattro indicatori di classi semantiche inferite (ThermalSymptom, MechanicalSymptom, ElectricalSymptom, HydraulicSymptom) e, per ciascuna delle quattro famiglie di guasto, il supporto dei sintomi presenti, il numero di sintomi associati ma dichiarati assenti, la compatibilità con la macchina e il supporto condizionato alla compatibilità.

Questo processo è un semantic lifting: dati descritti con feature di basso livello vengono integrati con attributi che esprimono relazioni concettuali della base di conoscenza. L'arricchimento non garantisce un aumento delle prestazioni: può introdurre ridondanza, correlazioni o segnali non utili a un dato classificatore.

4.3 Pre-processing dello Spazio delle Feature: Standardizzazione e Codifica Nominale

Le feature numeriche vengono convertite in valori numerici e i valori non interpretabili diventano mancanti. Le categorie nominali sono codificate mediante indicatori one-hot: una colonna per ciascun tipo di macchina e per

ciascuno stato ammesso dei sensori e dei sintomi. I valori mancanti o non riconosciuti negli stati vengono normalizzati nello stato SCONOSCIUTO, secondo le regole del trasformatore.

La pipeline scikit-learn applica l'imputazione mediana e la standardizzazione a tutte le colonne della rappresentazione in ingresso, incluse le feature numeriche, gli indicatori one-hot e le feature semantiche OntoBK. Questa scelta è più rilevante per SVM e MLP, ma viene mantenuta anche per Random Forest nella configurazione implementata. Pressione e portata, non applicabili a motori e nastri, risultano mancanti e vengono imputate con la mediana del training fold; la distinzione rispetto alle misure reali resta disponibile tramite gli indicatori one-hot degli stati dei sensori. Imputazione e standardizzazione sono apprese all'interno di ciascun fold di training, così da limitare la contaminazione fra training e validazione. Le rappresentazioni Base e OntoBK sono confrontate con la stessa procedura di cross-validation.

4.4 Il Modello Predittivo Ensemble: Random Forest

Random Forest combina le predizioni di un insieme di alberi decisionali addestrati su campioni e sottoinsiemi di feature. La combinazione tende a ridurre la varianza rispetto a un singolo albero e può modellare relazioni non lineari fra misure e sintomi. La configurazione del progetto usa 200 alberi e class_weight="balanced"; poiché il dataset ha lo stesso numero di esempi per ciascuna classe, l'opzione non risponde a uno squilibrio osservato ma fa parte della configurazione effettivamente valutata. Il modello viene valutato sia sulle feature Base sia sulle feature OntoBK, mantenendo uguale la procedura sperimentale.

Nei risultati registrati, Random Forest raggiunge accuracy 0,9032 con la rappresentazione Base e 0,8752 con OntoBK. In questo esperimento l'arricchimento semantico non migliora il modello: la differenza è di −2,80 punti percentuali. Il risultato evidenzia che feature semantiche ridondanti o non sufficientemente informative possono peggiorare le prestazioni, anche se sono concettualmente interpretabili.

4.5 Classificazione a Massimizzazione del Margine: Support Vector Machines (SVM)

La Support Vector Machine cerca una separazione a margine massimo fra classi; il kernel RBF permette di rappresentare confini non lineari nello spazio originale delle feature. La standardizzazione è importante perché le distanze utilizzate dal modello sono sensibili alla scala delle variabili. Anche per la SVM è impostato class_weight="balanced", mantenuto per descrivere fedelmente il modello implementato. La SVM è valutata nelle stesse condizioni dei modelli concorrenti.

L'accuracy osservata è 0,9084 per Base e 0,8981 per OntoBK, con una differenza di −1,03 punti percentuali. Anche in questo caso non emerge un vantaggio dell'arricchimento. Il dato va letto nel contesto del dataset sintetico e della specifica configurazione del modello, non come conclusione generale contro l'uso delle ontologie.

4.6 Il Paradigma Connessionista: Reti Neurali Artificiali (Multi-Layer Perceptron)

Il Multi-Layer Perceptron (MLP) è una rete neurale feed-forward che apprende trasformazioni non lineari tramite strati di neuroni e una funzione di perdita supervisionata. La configurazione implementata utilizza due strati nascosti di 50 e 25 unità e un limite di 500 iterazioni, con early stopping basato su una validazione interna. Il pre-processing numerico aiuta l'ottimizzazione e rende le scale delle feature più confrontabili. I valori sono configurazioni fissate e non il risultato di una ricerca degli iperparametri; lo stesso vale per Random Forest e SVM RBF, senza affermare che siano ottimali.

L'MLP ottiene accuracy 0,9139 su Base e 0,9146 su OntoBK, la migliore coppia di risultati fra quelle registrate. Il vantaggio OntoBK è tuttavia molto piccolo: circa +0,067 punti percentuali. La differenza è insufficiente per

affermare, senza ulteriori esperimenti e analisi d'incertezza, che l'arricchimento semantico migliori in modo sostanziale la generalizzazione.

4.7 Valutazione Statistica Rigorosa e Analisi Comparativa delle Prestazioni

Il confronto utilizza RepeatedStratifiedKFold con 5 fold e 3 ripetizioni, seed 42. La stratificazione mantiene approssimativamente le proporzioni delle classi nei fold. Ogni configurazione viene valutata sulle stesse partizioni, così da rendere più interpretabile il confronto fra Base e OntoBK. Le metriche salvate comprendono accuracy, precision macro, recall macro e F1 macro, oltre alla deviazione standard (calcolata sui 15 fold, con denominatore N). Le matrici di confusione aggregano le predizioni out-of-fold delle ripetizioni e quindi contengono 9.000 predizioni complessive per configurazione, 2.250 per classe; poiché i fold hanno uguale dimensione, l'accuracy ricavata da ciascuna matrice coincide con l'accuracy media riportata in tabella.

Modello	Rappresentazione	Accuracy ± std	F1 macro ± std	Δ accuracy OntoBK
Random Forest	Base	0,9032 ± 0,0142	0,9030 ± 0,0146	—
Random Forest	OntoBK	0,8752 ± 0,0142	0,8747 ± 0,0144	−2,80 p.p.
SVM RBF	Base	0,9084 ± 0,0100	0,9082 ± 0,0100	—
SVM RBF	OntoBK	0,8981 ± 0,0120	0,8978 ± 0,0121	−1,03 p.p.
MLP	Base	0,9139 ± 0,0126	0,9140 ± 0,0126	—
MLP	OntoBK	0,9146 ± 0,0098	0,9144 ± 0,0099	+0,067 p.p.

I risultati mostrano un quadro non uniforme. Base supera OntoBK per Random Forest e SVM; per MLP la versione OntoBK è lievemente migliore, ma il vantaggio è minimo rispetto alla variabilità osservata. Questo è un risultato utile: l'integrazione simbolico-statistica deve essere valutata sperimentalmente, non assunta come automaticamente benefica.

Le metriche riportate sono riferite al dataset sintetico generato dal progetto. Le deviazioni standard descrivono la variabilità delle valutazioni ripetute, ma non equivalgono a un intervallo di confidenza indipendente e non eliminano la necessità di una validazione esterna su dati reali. La ripetizione dei fold implica inoltre che le 9.000 predizioni aggregate non sono 9.000 casi indipendenti: rappresentano tre valutazioni ripetute degli stessi 3.000 casi.

4.8 Diagnosi simbolica, interpretabilità e gestione delle evidenze

La versione corrente non implementa apprendimento online, aggiornamento adattativo dei modelli o un feedback loop automatico. Per questo la sezione descrive il contributo della diagnosi simbolica e il trattamento dell'incertezza osservativa.

L'interfaccia testuale consente di selezionare il tipo di macchina e assegnare a ciascun sintomo uno stato: presente, assente, sconosciuto o non applicabile. Gli stati sconosciuti e non applicabili non vengono conteggiati come evidenza favorevole o contraria. I sintomi associati a un guasto ma dichiarati assenti vengono riportati separatamente, offrendo un elemento di spiegazione qualitativa, senza ridurre il supporto. Le ipotesi incompatibili con la macchina selezionata sono escluse prima del ranking.

La spiegazione rimane circoscritta alle evidenze conteggiate e alle relazioni codificate nell'ontologia. Il sistema non fornisce una probabilità calibrata né una stima dell'incertezza epistemica; inoltre, il conteggio simbolico non sostituisce una diagnosi di sicurezza o il giudizio di un manutentore esperto.

Capitolo 5 – Conclusioni e Sviluppi Futuri

5.1 Sintesi dei Risultati Raggiunti

Il progetto realizza un prototipo KBS che combina rappresentazione OWL, inferenza automatica, diagnosi simbolica e confronto di modelli supervisionati. L'ontologia definisce classi, relazioni, compatibilità e assiomi per la classificazione dei sintomi. HermiT esegue il ragionamento e DiagnosticReasoner rende accessibili le classi inferite; la diagnosi simbolica ordina le ipotesi usando le relazioni dell'ontologia e le evidenze fornite. Il modulo OntoBK costruisce una rappresentazione arricchita e consente un confronto sperimentale con la rappresentazione Base.

La valutazione su 3.000 casi sintetici ha prodotto accuracy fra circa 0,875 e 0,915 nelle configurazioni esaminate. L'MLP presenta la prestazione più alta nei risultati registrati; l'aggiunta di feature semantiche produce però un beneficio trascurabile solo per questo modello e riduce l'accuracy di Random Forest e SVM. Il progetto mostra dunque sia un'integrazione tecnica fra KBS e ML sia l'importanza di una valutazione empirica critica.

5.2 Valutazione Critica dell'Architettura

I punti di forza sono la separazione fra conoscenza generale e dati tabellari, la possibilità di ispezionare le relazioni ontologiche, la gestione esplicita degli stati mancanti e sconosciuti, la riproducibilità della generazione sintetica e il confronto di più modelli con la stessa strategia di validazione. I test automatici contribuiscono a verificare le funzioni principali e i casi limite implementati.

I limiti principali sono la natura sintetica dei dati, la semplificazione delle famiglie di guasto, il numero contenuto di concetti OWL e la natura euristica del ranking simbolico. Il dataset è generato da regole e distribuzioni definite nel codice: ciò può rendere più facile per i classificatori apprendere i pattern artificiali e limita la generalizzazione a condizioni operative non simulate. Inoltre, l'arricchimento OntoBK non produce miglioramenti sistematici e richiede una progettazione attenta delle feature semantiche.

La diagnosi simbolica e i classificatori ML svolgono ruoli distinti. La prima privilegia trasparenza e compatibilità basata su regole; i secondi apprendono regolarità statistiche dai casi. Nessuno dei due è da solo una soluzione certificata per la manutenzione industriale. L'architettura costituisce una base sperimentale didattica e un punto di partenza per ulteriori validazioni.

5.3 Sviluppi Futuri

1. Sostituire o affiancare il dataset sintetico con dati industriali reali, documentati e raccolti in condizioni operative diversificate.
2. Estendere l'ontologia con sottotipi di guasto, componenti, relazioni causali, vincoli di cardinalità pertinenti e una modellazione più completa delle osservazioni.
3. Raccogliere valutazioni da esperti per verificare la correttezza delle relazioni sintomo-guasto e delle compatibilità macchina-guasto.
4. Analizzare le feature semantiche con ablation study, selezione delle feature e confronti statisticamente più approfonditi.
5. Valutare modelli probabilistici o metodi di calibrazione per fornire stime d'incertezza distinte dal supporto simbolico.
6. Progettare, come estensione separata e solo dopo aver raccolto dati aggiornati, un meccanismo di feedback umano e apprendimento incrementale con controllo di versione e validazione prima della messa in esercizio.
7. Migliorare l'interfaccia con visualizzazione del grafo, tracciamento delle prove e report diagnostici esportabili.

Appendice A – Struttura essenziale del progetto

Percorso	Responsabilità
src/ontology.py	Costruzione OWL e riepilogo della base di conoscenza.
src/semantic_reasoner.py	Inferenza, cache semantica e diagnosi simbolica.
src/ontoBK_learning.py	Estrazione feature, pipeline ML e valutazione comparativa.
src/dataset_generator.py	Generazione riproducibile del dataset sintetico.
src/main.py	Interfaccia testuale e coordinamento delle funzioni.
data/industrial_faults.csv	Dataset sintetico utilizzato negli esperimenti.
results/industrial_diagnosis.owl	Esportazione dell'ontologia.
results/comparison_results.csv	Metriche aggregate dei confronti Base/OntoBK.
results/confusion_*.csv	Matrici di confusione dei modelli e delle rappresentazioni.
tests/	Test automatici per generatore, ontologia, reasoner e feature ML.

Appendice B – Riproducibilità e interpretazione dei risultati

Dalla directory principale del progetto, installare le dipendenze indicate in requirements.txt. L'esecuzione di python -m src.main apre il menu testuale. La diagnosi simbolica richiede la costruzione dell'ontologia e l'esecuzione del reasoner HermiT; la valutazione ML utilizza data/industrial_faults.csv e salva i risultati nella directory results. L'esecuzione del reasoner richiede un ambiente Java compatibile con Owlready2/HermiT.

```
python -m pip install -r requirements.txt
python -m src.main
python -m pytest
```

I risultati in questa relazione corrispondono ai valori registrati nel file results/comparison_results.csv incluso nella versione esaminata. Prima della consegna finale è opportuno rieseguire la suite di test nell'ambiente di destinazione e aggiornare i risultati se cambia il codice, il dataset o la configurazione sperimentale.